

MINOR ASSIGNMENT-04

Practical Programming with C (CSE 3544)

Publish on: 04-11-2025

Course Outcome: CO₃

Program Outcome: PO₃

Submission on: 11-11-2025

Learning Level: L₄

Problem Statement:

Working with pointers, *referencing* a variable through a pointer and accessing the contents of a memory cell through a pointer variable that stores its address (*i.e. indirect reference*).

Assignment Objectives:

To learn about pointers, referencing, indirect referencing and how to return function results through a function's parameters (input parameters, input/output parameters, output parameters). Also to understand the differences between call-by value & call-by-reference.

1. For the given structure below, declare the variable type, and print their values as well as addresses;



Space for Program ▼

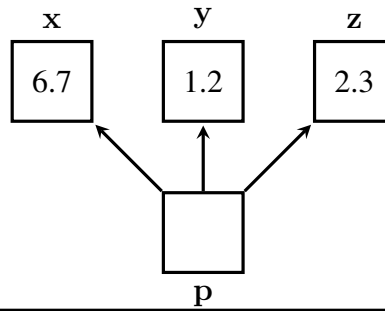
Output ▼

2. Declare two integer variable and assign values to them, and print their addresses. Additionally, Swap the contents of the variables and print their addresses after swap. State whether the addresses before and after are equal or not.

Space for Program ▼

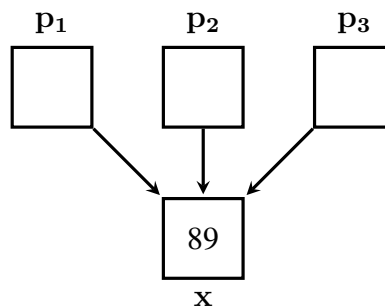
Output ▼

3. Write the C statement to declare and initialize the pointer variable, **p**, for the given structure and display the values of **x**, **y** and **z** with the help of **p**.



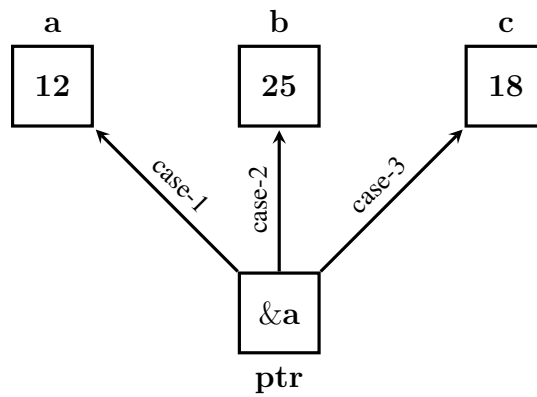
Space for Program ▼	Output ▼

4. Write the C statement to declare and initialize the pointer variables `p1`, `p2` and `p3` for the given structure and display the value of `x` from `p1`. Also update the value of `x` to 100 using pointer `p3`.



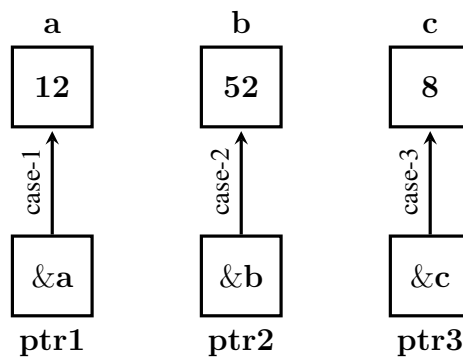
Space for Program ▼	Output ▼

5. Write the C statement to declare and initialize the pointer variable for the given structure and update the values of `a`, `b` and `c` to be incremented by 10 through the pointer variable.



Space for Program ▼	Output ▼

6. Write the C statement to declare and initialize the pointer variables for the given structure and update the values of a, b and c to be incremented by 10 through their respective pointers.



Space for Program ▼	Output ▼

-
- ```

graph LR
 ptr1[ptr1] --> a_ptr[&a]
 a_ptr --> a_val[52]
 ptr2[ptr2] --> b_ptr[&b]
 b_ptr --> b_val[18]

```

| Space for Program ▼ | Output ▼ |
|---------------------|----------|
|                     |          |

- |   |    |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|----|
| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
| 0 | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 |
- `&a[0]&a[1]&a[2]&a[3]&a[4]&a[5]&a[6]&a[7]&a[8]&a[9]`

Space for Program ▼

Output ▼

9. Declare the two arrays to hold the values as shown in the given rectangular boxes. Write the equivalent C statement to print their values and addresses through pointer (**Hint**: an array name is a pointer to the first element in the array).

|    |    |    |    |    |
|----|----|----|----|----|
| 10 | 13 | 20 | 33 | 44 |
|----|----|----|----|----|

|      |      |      |      |      |      |
|------|------|------|------|------|------|
| 10.2 | 13.3 | 20.0 | 33.3 | 45.3 | 89.9 |
|------|------|------|------|------|------|

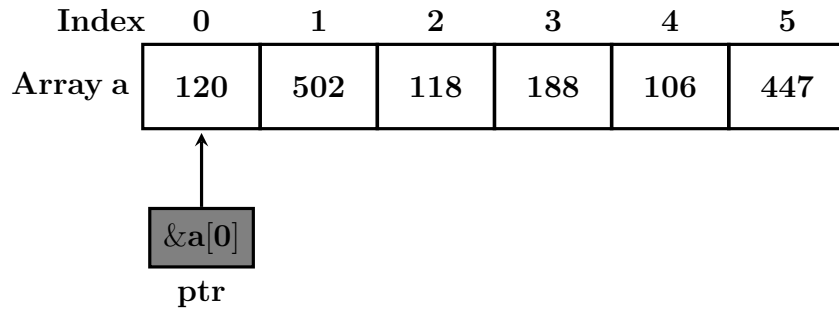
| Space for Program ▼ | Output ▼ |
|---------------------|----------|
|                     |          |

10. Write the C statement to declare and initialize the pointer variables for the given structure and display the array content using pointer.

|         |       |       |       |       |       |       |
|---------|-------|-------|-------|-------|-------|-------|
| Index   | 0     | 1     | 2     | 3     | 4     | 5     |
| Array a | 120   | 502   | 118   | 188   | 106   | 447   |
|         | ↑     | ↑     | ↑     | ↑     | ↑     | ↑     |
|         | &a[0] | &a[1] | &a[2] | &a[3] | &a[4] | &a[5] |
|         | ptr1  | ptr2  | ptr3  | ptr4  | ptr5  | ptr6  |

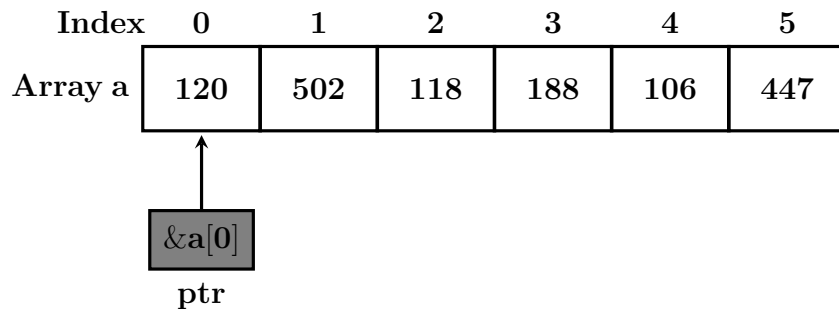
| Space for Program | Output ▼ |
|-------------------|----------|
|                   |          |

11. Write the C statement to declare and initialize the pointer variable for the given structure and display the array content using the pointer, **ptr**.



| Space for Program | Output ▼ |
|-------------------|----------|
|                   |          |

12. As array name is a pointer, so modify the assignment **ptr=a** rather **ptr=&a[0]**. Write the C statement to declare and initialize the pointer variable for the given structure and display the array content using pointer, **ptr**.



| Space for Program | Output ▼ |
|-------------------|----------|
|                   |          |

13. Trace the execution of the following fragment at **line-1**.

```
int m = 10, n = 5;
int *mp, *np;
mp = &m;
np = &n;
*mp = *mp + *np;
*np = *mp - *np;
printf("%d %d\n%d %d\n", m, *mp, n, *np); /*
 line-1 */
```

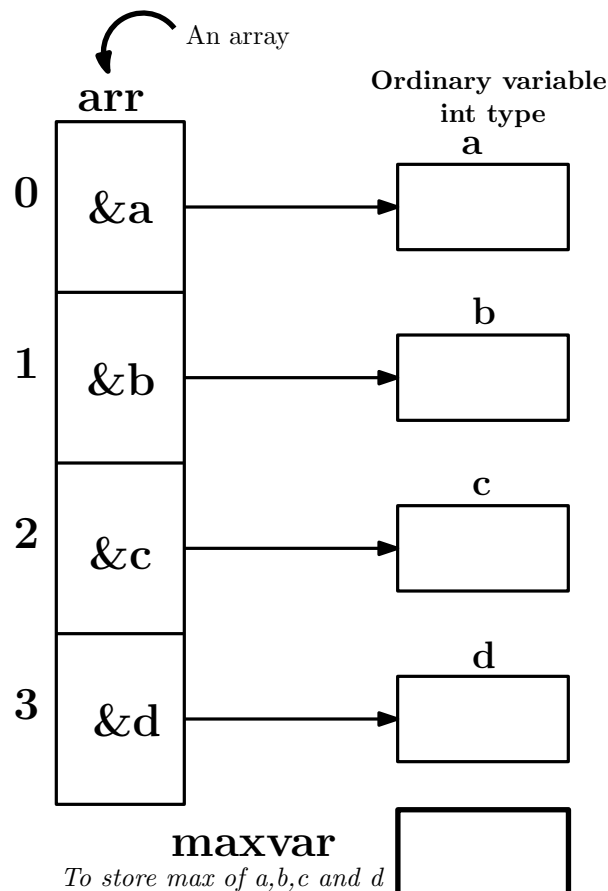
Output▼

14. Given the declarations;

```
int m = 25, n = 77;
char c = '*';
int *itemp;
/* describe the errors in each of the
following statements. */
m = &n;
itemp = m;
*itemp = c;
*itemp = &c;
```

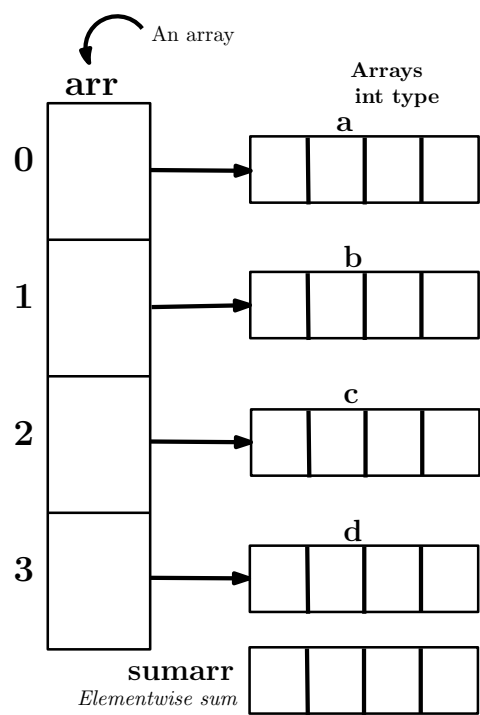
Output▼

15. Simulate the following structure in C to store **55** in **a**, **105** in **b**, **89** in **c** and **68** in **d** using their respective pointers. Additionally, find the maximum among **a**, **b**, **c** and **d** through pointer manipulation. Finally Store the maximum to the required variable and display the maximum.



C simulation▼

16. Simulate the following structure in C to find the element sum of the given arrays **a**, **b**, **c** and **d** into **sumarray** using their respective pointers. The 1-D arrays must be read/scanned through the pointers.

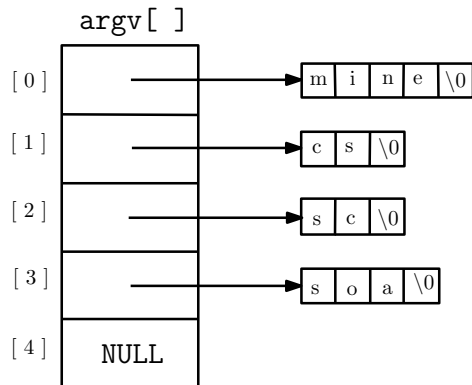


C simulation▼

Test case: Input & Output▼



17. An argument array is an array of pointers to strings. The end of the array is marked by an entry containing a `NULL` pointer as shown in the figure. Write a C Simulation to implement the following figure and manipulate the character array to hold all capital case letters using pointer. Finally display the strings.



#### C simulation▼

#### Test case: Input & Output▼

18. Consider the following figures 1, 2 and 3 to manipulate the ordinary variables, integer arrays and strings through pointers. There exist no names associated with the variables, arrays and strings. State the method to allocate memory for the pointers to manipulate the desired variables.

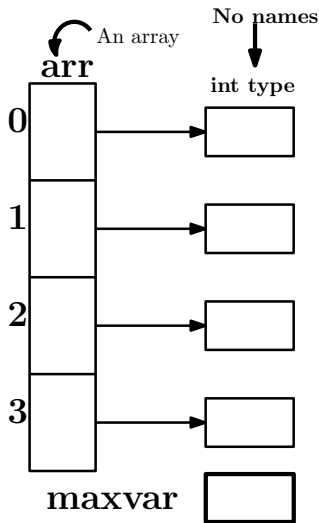


Figure-1

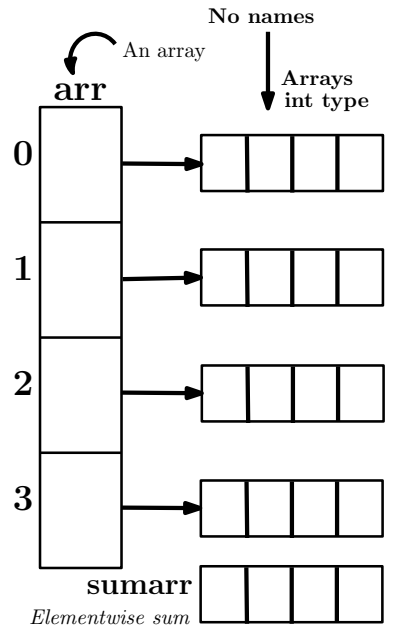


Figure-2

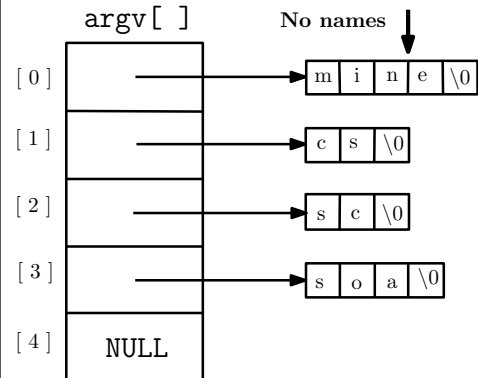
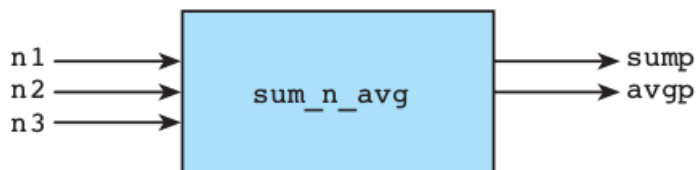


Figure-3

Answer ▼

19. Write a prototype for a function **sum\_n\_avg** that has three type double input parameters and two output parameters. The function computes the sum and the average of its three input arguments and relays its results through two output parameters.



Output ▼

20. The following code fragment is from a function preparing to call **sum\_n\_avg** (see question-19). Complete the function call statement.

```
double one, two, three, sum_of_3, avg_of_3;
printf("Enter three numbers> ");
scanf("%lf%lf%lf", &one, &two, &three);
sum_n_avg(____);
. . .
```

Define the function **sum\_n\_avg** whose prototype you wrote in question-19.

| Space for Program | Output ▼ |
|-------------------|----------|
|                   |          |

21. Write a program to use the idea of **multiple calls to a function with input/output parameters** to sort 6 integer numbers in ascending order without using any sorting algorithms. The prototype of the function to be used in your program to sort the numbers is given as **void arrange(int \*, int \*)**; and also draw the data areas of calling function and **arragne()** function for the first function call **arrange(....)**.

### Sample Run

```
printf("Enter SIX numbers separated by blanks> ");
12 3 56 8 20 654
/* Displays results */
printf("The numbers in ascending order are: %d %d %d %d %d %d\n", n1,
 n2, n3, n4, n5, n6);
3 8 12 20 56 654
```

| Space for Program ▼ | Output ▼ |
|---------------------|----------|
|                     |          |

22. Show the table of values for x, y, and z that is the output displayed by the following program.

```
#include <stdio.h>
void sum(int a, int b, int *cp);
int main(void){
 int x, y, z;
 x = 7; y = 2;
 printf("x y z\n\n");
 sum(x, y, &z);
 printf("%4d%4d%4d\n", x, y, z);
 sum(y, x, &z);
 printf("%4d%4d%4d\n", x, y, z);
 sum(z, y, &x);
 printf("%4d%4d%4d\n", x, y, z);
 sum(z, z, &x);
 printf("%4d%4d%4d\n", x, y, z);
 sum(y, y, &y);
 printf("%4d%4d%4d\n", x, y, z);
 return (0);
}

void sum(int a, int b, int *cp){
 *cp = a + b;
}
```

Output▼

23. (a) What values of x and y are displayed by this program? ( Hint: Sketch the data areas of **main**, **trouble**, and **double\_trouble** as the program executes.)

```
void double_trouble(int *p, int y);
void trouble(int *x, int *y);
int main(void){
 int x, y;
 trouble(&x, &y);
 printf("x = %d, y = %d\n", x, y);
 return (0);
}
void double_trouble(int *p, int y){
 int x;
 x = 10;
 *p = 2 * x - y;
}
void trouble(int *x, int *y){
 double_trouble(x, 7);
 double_trouble(y, *x);
}
```

Output▼

- (b) Classify each formal parameter of **double\_trouble** and **trouble** as input, output, or input/output.

Formal parameter classification ▼

24. Develop a program to reverse a string using pointer manipulation.

| Space for Program ▼ | Output ▼ |
|---------------------|----------|
|                     |          |

25. Design a program to find the largest value in an 1-D array using pointers.

Space for Program ▼

Output ▼

26. Design a program to transpose of a matrix using pointers.

| Space for Program ▼ | Output ▼ |
|---------------------|----------|
|                     |          |